

ShopLab local

Application FastAPI/PostgreSQL derrière Nginx, prévue exclusivement pour les TP sur loopback. Deux modes existent : `vulnerable` pour observer une cause bornée et `corrected` pour vérifier un contrôle. Le changement de mode n'est pas le travail de correction demandé dans les starters.

Cycle de vie

```
./scripts/lab-start.sh core
./scripts/health-check.sh core
./scripts/set-mode.sh corrected
./scripts/verify-lab.sh basic
./scripts/compose.sh --profile core ps
./scripts/reset.sh
./scripts/cleanup.sh
```

Les identifiants synthétiques sont `alice / atelier-alice`, `bob / atelier-bob`, `admin / atelier-admin`. Ne les réutilisez nulle part et ne les joignez pas à une remise. Les ports par défaut sont `127.0.0.1:8080` et `127.0.0.1:8443`; utilisez `SHOPLAB_HTTP_PORT` et `SHOPLAB_HTTPS_PORT` si nécessaire.

Le projet Compose est calculé depuis l'utilisateur et le dossier; surchargez avec un nom unique, par exemple `SHOPLAB_PROJECT=shoplab-groupe07`, pour des exécutions parallèles. Pour toute inspection directe, utilisez `./scripts/compose.sh ...` afin de viser ce même projet. Aucun `container_name` ni nom global de réseau n'est fixé.

Suites de vérification

`verify-lab.sh` accepte `basic`, `session`, `authz`, `injection`, `files`, `api`, `hardening`, `database` et `observability`. `database` vérifie la matrice de privilèges après réalisation du starter; `observability` conserve un WARN car l'interprétation des preuves reste guidée. `runtime-test.sh` possède un trap de cleanup et ne doit laisser aucune ressource, même après échec.

Limites connues

Sessions et quota sont en mémoire pour rester lisibles; production exigerait un magasin partagé. SSRF ne fetch jamais. TLS utilise un certificat jetable auto-signé. Les données et mots de passe sont synthétiques. Le profil `observe` demande davantage de mémoire et un premier téléchargement d'images.